Query Scheduling Optimization in IVF-based Vector Search

Yujia Qian — yjqian19@mit.edu
Xiangyu Guan — xiang949@mit.edu

Abstract

Vector search has become a foundational building block for AI applications such as retrieval-augmented generation and semantic search. To serve these workloads efficiently, most optimizations target the index structure. In contrast, we focus on the execution layer: given an IVF index, can we improve throughput by reorganizing how concurrent queries are batched and dispatched? We study three strategies—sequential execution, time-window batching, and cluster-based batching—and compare their trade-offs in latency, throughput, and hardware efficiency on a single machine.

Introduction

Vector search is a core component of modern AI applications. Most optimization efforts target indexing structures (e.g., HNSW, IVF variants), but less attention has been paid to how queries are scheduled at runtime.

This project asks: **can we improve performance by reorganizing query execution, without changing the index?**

We build on IVF (Inverted File Index), which partitions vectors into clusters (inverted lists). Each query probes only a few lists, and different queries often probe overlapping ones. This means queries can share loaded data if executed together.

Batching does not reduce total computation — the work stays at roughly `number of queries × vectors scanned`. But it improves how that work is organized:

- **Less redundant data access** — shared inverted lists are loaded once
- **Better cache locality** — data stays in cache across queries
- **SIMD parallelism** — multiple distance computations are vectorized together

We compare three strategies: sequential execution (baseline), time-window batching (group by arrival time), and cluster-based batching (group by shared inverted lists), evaluating latency, throughput, and efficiency on a single machine.

Methodology

Sequential Execution

Each query is processed independently: it finds the nearest centroids, scans the corresponding inverted lists, computes distances, and returns the top-k results. This serves as the baseline with no cross-query optimization.

Time-window Batching

Queries arriving within a fixed time window Δt are collected into a batch and executed together as a single call to the index. This allows multiple queries to share vectorized distance computation over the same inverted lists without any knowledge of query content — queries are grouped purely by arrival time.

The core trade-off is that a larger Δt produces larger batches with better hardware utilization, but increases how long each query waits before execution. To bound this, the window uses a dual-trigger flush: whichever fires first between the time limit Δt and a maximum batch size `max_batch_size`. Using only a time trigger risks tiny batches at low QPS and unbounded memory at high QPS; using only a size trigger risks indefinite waiting at low QPS. We simulate query arrivals via a Poisson process at a target QPS λ and sweep Δt ∈ {0.5–50} ms and `max_batch_size` ∈ {32–256} to find the optimal operating point on the latency–throughput curve.

Cluster-based Batching

Queries are grouped by the inverted lists they access. Queries that probe overlapping clusters are placed in the same batch so that each list is loaded and scanned once, then reused across all queries in the group. This strategy is more selective than time-window batching but requires an extra grouping step.

Evaluation

- Latency (average and tail, e.g. P95)
- Throughput (queries per second)
- Precision / recall@*k*

We expect cluster-based batching to yield the highest throughput under clustered query workloads where probe-list overlap is high, while time-window batching should offer a better latency-throughput trade-off under random workloads where the grouping overhead outweighs the sharing benefit.

Data

We use two standard ANN benchmark datasets:

- **SIFT1M** (primary) — 1,000,000 128-dimensional SIFT descriptors with 10,000 queries and precomputed ground truth. Evaluated with L2 distance. Well-separated cluster geometry makes it the standard reference for IVF benchmarking; our numbers are directly comparable to prior work.
- **GloVe-100** (secondary) — 1,183,514 100-dimensional word embeddings with 10,000 queries. Evaluated with cosine similarity (inner product on normalized vectors). More diffuse cluster structure than SIFT, used to validate that results generalize beyond clean geometric data.

Workloads include:

- **Random queries** — queries sampled uniformly; low probe-list overlap across queries. Worst case for cluster-based batching.
- **Clustered queries** — queries drawn from a few regions of the vector space; high overlap. Best case for cluster-based batching.

All data resides in memory. No disk I/O during search.

Tasks

- **Core engine:** IVF index, in-memory data, and correct single-query search as the baseline. Implemented using Faiss (`IndexIVFFlat`), which provides the reference IVF implementation with k-means training, centroid lookup, and SIMD-accelerated distance computation.
- **Schedulers:** Sequential, time-window, and cluster-based execution paths over the same engine.
- **Evaluation:** Benchmark workloads, metrics, and comparison plots.

Timeline

Phase 1: Now → Apr 14, Mid-term (2 weeks)

- Build IVF-based retrieval system
- Load dataset and benchmark
- Validate single-query execution

Phase 2: Apr 14 → May 3 (3 weeks)

- Implement scheduling strategies
- Iteratively refine
- Run experiments and compare performance

Phase 3: May 3 → May 7 (1 week)

- Compose the report and prepare the video

Deliverables

- A working system
- Measured performance differences
- A clear explanation of results